

Relatório 8: Avaliação de performance com e sem cache blocking

João Bruno dos Santos, João Lucas Medina Kormann

Universidade Federal de Santa Catarina

I. INTRODUÇÃO

Este relatório tem o objetivo de apresentar a metodologia aplicada para a resolução dos problemas do Laboratório 8 da disciplina Organização de Computadores I, ministrada pelo professor Marcelo Daniel Berejuck, utilizando a linguagem Assembly para a arquitetura MIPS (Microprocessor without Interlocked Pipelined Stages), o simulador MARS (MIPS Assembler and Runtime Simulator).

O intuito principal é analisar o comportamento do cache de dados durante a execução de diferentes implementações de manipulação de matrizes com tamanho variável. Para isso, foi empregada a ferramenta Data Cache Simulator, permitindo observar e comparar métricas como o Cache Miss Rate. Através dessa análise, busca-se compreender como o acesso à memória influencia o desempenho do sistema.

II. IMPLEMENTAÇÃO E DISCUSSÃO DO ALGORITMO 1

A. PASSOS INICIAIS

A implementação do algoritmo 1 foi realizada de maneira simples, sem utilizar, em geral, nenhum conceito explorado em algum laboratório anterior. Vale ressaltar que, como foi necessário utilizar um valor parametrizável para MAX, foi utilizado uma constante com a notação `.eqv`, assim alterando seu valor atualizando somente uma linha de código.

Para os valores de A e B na Figura 1, obtivemos a matriz resultante na Figura 2.

```
.eqv    MAX 4    # Constante do tamanho da matriz

.data
A:      .float 1.0, 2.0, 3.0, 4.0,
        5.0, 6.0, 7.0, 8.0,
        9.0, 10.0, 11.0, 12.0,
        13.0, 14.0, 15.0, 16.0

B:      .float 10.0, 20.0, 30.0, 40.0,
        50.0, 60.0, 70.0, 80.0,
        90.0, 100.0, 110.0, 120.0,
        130.0, 140.0, 150.0, 160.0
```

Figura 1 - Matrizes A e B de entrada

```
-- program is finished running --

11.0 52.0 93.0 134.0
25.0 66.0 107.0 148.0
39.0 80.0 121.0 162.0
53.0 94.0 135.0 176.0
```

Figura 2 - Resultado obtido exibido no terminal

B. ANÁLISE DE DESEMPENHO

Para os testes de desempenho do algoritmo, foi utilizada a política de *Direct Mapping*, com 8 blocos de tamanho 4 palavras. Alterando o valor de MAX, os resultados obtidos são os exibidos na Tabela 1.

Valor de MAX	Cache Hit Rate
4	83%
8	54%
16	54%
32	54%

Tabela 1 - Performance do algoritmo 1

Com esses resultados, é possível perceber que o algoritmo possui um desempenho bom enquanto o cache consegue armazenar os dados para o cálculo.

Como o acesso da matriz B quebra a localidade espacial e o algoritmo não está preparado para isso, os dados são substituídos continuamente, causando uma queda drástica de desempenho.

III. IMPLEMENTAÇÃO E DISCUSSÃO DO ALGORITMO 2

A. PASSOS INICIAIS

Assim como no caso do Algoritmo 1, a implementação do algoritmo 2 foi básica também, sendo necessária apenas a “tradução” do código em C proposto pelo laboratório para assembly. Para tal, foi reutilizado o próprio código do algoritmo 1 expandindo-o para que suportasse mais 2 loops, responsáveis pela execução dos blocos separados dentro das matrizes.

Também de forma semelhante ao caso anterior, a fim de tornar o código parametrizável, utilizamos *.eqv* nos parâmetros *MAX* e *block_size* para que assim fosse necessário alterar apenas uma vez seu valor.

```
.eqv MAX 4
.eqv block_size 4    # pode alterar para 1, 2, 4, etc.

.data
A: .float 1.0, 2.0, 3.0, 4.0,
    5.0, 6.0, 7.0, 8.0,
    9.0, 10.0, 11.0, 12.0,
    13.0, 14.0, 15.0, 16.0

B: .float 10.0, 20.0, 30.0, 40.0,
    50.0, 60.0, 70.0, 80.0,
    90.0, 100.0, 110.0, 120.0,
    130.0, 140.0, 150.0, 160.0
```

Figura 3 - Parâmetros iniciais Algoritmo 2

```
for_i:
    bge $t1, $s0, end_i

    # limite_i = min(i + block_size, MAX)
    add $t5, $t1, block_size
    blt $t5, $s0, no_adj_i
    move $t5, $s0
no_adj_i:

    li $t2, 0          # j = 0
for_j:
    bge $t2, $s0, end_j

    # limite_j = min(j + block_size, MAX)
    add $t6, $t2, block_size
    blt $t6, $s0, no_adj_j
    move $t6, $s0
no_adj_j:

    move $t3, $t1      # ii = i
for_ii:
    bge $t3, $t5, end_ii

    move $t4, $t2      # jj = j
```

Figura 4 - Código inicial Algoritmo

Por fim, como outro ponto interessante de ser citado, temos a utilização do *label* “*no_adj*” que serve para limitar o tamanho de *i* e de *j* até *MAX*, pois como somamos *block_size* em ambos, caso o tamanho de *block_size* não seja bem pensado, é possível que *i* e *j* ultrapassassem *MAX*, fazendo assim com que fossem acessados espaços fora do tamanho máximo das linhas e colunas definidos previamente.

B. ANÁLISE DE DESEMPENHO

Para os testes de desempenho do algoritmo 2, foi utilizada a política de *Direct Mapping*, com 8 blocos de tamanho 4 palavras e *block_size* igual a 4. Alterando o valor de *MAX*, os resultados obtidos são os exibidos na Tabela2.

Valor de MAX	Cache Hit Rate
4	86%
8	80%
16	70%
32	70%

Tabela 2 - Performance do algoritmo 2

Com este resultado, percebe-se a eficiência do método de cache blocking pois mesmo com valores maiores que o tamanho do cache, ainda é mantido um *Hit Rate* considerável até para os casos onde a matriz é extremamente maior que o valor de cache, como no caso do $MAX = 32$, onde o valor em bytes seria 4096 para cada matriz, totalizando 32x o tamanho da cache.

IV. CONSIDERAÇÕES FINAIS

De modo a comparar os resultados, as taxas de acerto de cada algoritmo foram compiladas no Gráfico 1, onde podemos observar que o algoritmo otimizado (2) possui uma eficiência maior em todos os casos, tornando-se significativa quando o tamanho do cache já não é suficiente para comportar os dados que não se aproveitam da localidade espacial.

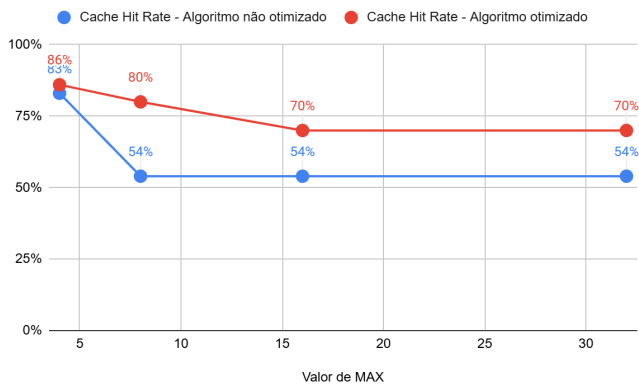


Tabela 2 - Comparação de performance entre algoritmo não otimizado (1) e algoritmo otimizado (2)

Portanto, percebe-se que a utilização de conceitos de localidade espacial se tornam altamente necessárias para a otimização de algoritmos em situações em que o tamanho da cache é limitado.